

MVP Versions

Presentation Confidence Detector

Project Blueprint

MVP Versions — What to Build and When

The Build Philosophy

Same as ExamGuard: build in layers. Each version is a WORKING product you can demo. Do not try to build everything at once.

v0.1	→	v0.2	→	v0.3	→	v1.0
Face		+ Speech		+ Full		+ Polish
Only		+ STT		System		+ AI API
1 week		1 week		2-3 weeks		2 weeks

Each version answers one question:

- v0.1: "Can I read a face?"
- v0.2: "Can I listen to speech?"
- v0.3: "Can I combine everything into one score?"
- v1.0: "Is this a product someone would pay for?"

Version 0.1 — Face Only

BUILD THIS AFTER: Phase 3 (Face & Expression Detection)

What This Version Does

One camera. One job. Detect facial expressions and show a face mesh overlay.

That is it. No microphone, no speech-to-text, no scoring algorithm. Just:

```
Camera → MediaPipe Face Mesh → Expression Label + Face Overlay
```

This is your FIRST working piece of the Confidence Detector. It proves one thing: **MediaPipe works in my browser and I can read facial expressions.**

Why Start Here

```
Full system with 4 engines: 3-4 weeks to build, many things can fail
Face mesh only:           3-5 days to build, works immediately

A simple working demo >> A complex unfinished project
```

Face-only is useful because:

- It visually proves the technology works (the mesh on your face is impressive)
- It validates your camera access pipeline (permissions, stream setup)
- Expression detection is the most visually dramatic engine
- You get instant feedback: smile and see the label change

Tech Stack

Component	Technology	Purpose
Camera input	getUserMedia API	Access webcam
Face detection	MediaPipe Face Mesh	468-point face mesh
Expression logic	Custom JS	Map landmarks to labels
UI	React + Vite	Display feed + overlay

No microphone. No audio. No backend. Just camera + face mesh + display.

What Phases Must Be Complete

Phase	Required?	Why
Phase 1: Hardware Access	YES	Camera permissions, video stream setup
Phase 2: Speech & Audio	No	No audio in v0.1
Phase 3: Face & Expression	YES	MediaPipe Face Mesh, expression mapping
Phase 4: NLP Text Analysis	No	No text in v0.1
Phase 5: System Integration	No	Only one engine, no integration needed

What You Will Learn Building This

1. **Browser camera access** — getUserMedia, permissions, stream lifecycle
2. **MediaPipe in the browser** — Loading the model, processing frames, reading landmarks
3. **Canvas overlay rendering** — Drawing the 468-point mesh on top of the video feed
4. **Expression mapping** — Translating landmark distances into emotion labels
5. **Real-time processing loop** — requestAnimationFrame, processing frames without freezing UI
6. **React video components** — Displaying a live camera feed in a React app

Step-by-Step Build Guide

Step 1: Set Up the Project (30 minutes)

You scaffold a new React project using Vite with the React template, then install the MediaPipe face mesh and camera utilities packages. The project structure has a `components/` folder (CameraFeed, FaceMeshOverlay, ExpressionLabel) and an `engines/` folder (faceEngine wrapper), keeping the UI and AI logic cleanly separated.

Step 2: Camera Access (1-2 hours)

Build a component that:

- Requests camera permission
- Shows the live video feed
- Handles permission denied gracefully

- Shows a "Camera loading..." state

Step 3: MediaPipe Face Mesh (2-4 hours)

Build the face engine:

- Initialize MediaPipe Face Mesh
- Process each video frame
- Extract the 468 facial landmarks
- Draw the mesh points on a canvas overlay

Step 4: Expression Detection (4-8 hours)

Map landmarks to expressions:

- Calculate mouth openness (landmark distances)
- Calculate eye openness
- Calculate eyebrow position
- Map combinations to labels: "Neutral", "Happy", "Concerned", "Surprised"
- Display the current expression label

Step 5: Basic UI (2-4 hours)

Put it together:

- Camera feed with mesh overlay
- Expression label below the feed
- Start/Stop button
- FPS counter (to verify performance)

Success Criteria

Metric	Target
Face mesh appears on face	Yes, 468 points visible
Expression label updates	Changes when you smile/frown
Frame rate	> 15 FPS
Camera permission handled	Graceful error if denied
Works in Chrome	Yes
Works in Firefox	Yes (stretch goal)

Limitations (What v0.1 Cannot Do)

Cannot do: Listen to speech, analyze text, measure voice pitch, calculate confidence score, show coaching alerts, generate reports, track multiple people

These will be added in v0.2, v0.3, and v1.0!

After You Build This

You have a working face mesh viewer with expression labels. It looks impressive and it proves the core technology works. Show it to someone — watching the mesh track their face in real time always gets a reaction.

Now move on to v0.2 to add the ears (speech recognition).

Version 0.2 — Face + Speech

BUILD THIS AFTER: Phase 4 (NLP & Text Analysis)

What This Version Does

Camera + microphone. Two engines. Face expressions AND live speech analysis, shown side by side.

```
Camera → Face Mesh → Face Score
Microphone → STT → Transcript → NLP Analysis → Speech Score
```

Two separate scores displayed. NOT combined yet. You are proving that both engines work independently before you try to merge them.

Why This Order

```
v0.1 proved:  "I can read a face"
v0.2 proves:  "I can read a face AND listen to speech AT THE SAME TIME"
```

```
The hardest part of v0.2 is not the NLP.
The hardest part is running two engines simultaneously without lag.
```

v0.2 is critical because:

- It is the first time you run two engines in parallel
- The live transcript with highlighted fillers is immediately useful
- Two separate scores let you verify each engine independently before combining
- You discover performance issues early (before adding engines 3 and 4)

Tech Stack

Component	Technology	Purpose
Camera input	getUserMedia API	Access webcam
Face detection	MediaPipe Face Mesh	468-point face mesh
Expression logic	Custom JS	Map landmarks → face score
Microphone	getUserMedia (audio)	Access microphone
Speech-to-Text	Web Speech API	Live transcript (Chrome/Edge MVP)
NLP Analysis	Custom JS (Phase 4)	Fillers, hedges, pace
UI	React + Vite	Two-panel dashboard

What Phases Must Be Complete

Phase	Required?	Why
Phase 1: Hardware Access	YES	Camera + microphone access
Phase 2: Speech & Audio	YES	Browser STT choice (Web Speech API for MVP) + audio stream
Phase 3: Face & Expression	YES	MediaPipe Face Mesh, expression scoring
Phase 4: NLP Text Analysis	YES	Filler detection, hedge detection, pace calculation
Phase 5: System Integration	Partially	Basic two-engine orchestration, but not full integration

What You Will Learn Building This

7. **Dual-engine orchestration** — Running face + speech engines simultaneously
8. **Microphone access** — `getUserMedia` for audio, handling permissions
9. **Web Speech API** — Real-time speech-to-text for the browser MVP (Chrome/Edge first)
10. **Live transcript rendering** — Displaying text as it is spoken, word by word
11. **Filler word highlighting** — Marking fillers in the transcript with color
12. **Two-panel UI** — Side-by-side layout for face and speech data
13. **Independent scoring** — Calculating face score and speech score separately

Step-by-Step Build Guide

Step 1: Add Microphone Access (1-2 hours)

Extend the hardware layer:

- Request microphone permission alongside camera
- Handle "camera yes, mic no" and "camera no, mic yes" scenarios
- Show mic level indicator (simple volume bar)

Step 2: Add Speech-to-Text Engine (2-4 hours)

Build the STT engine:

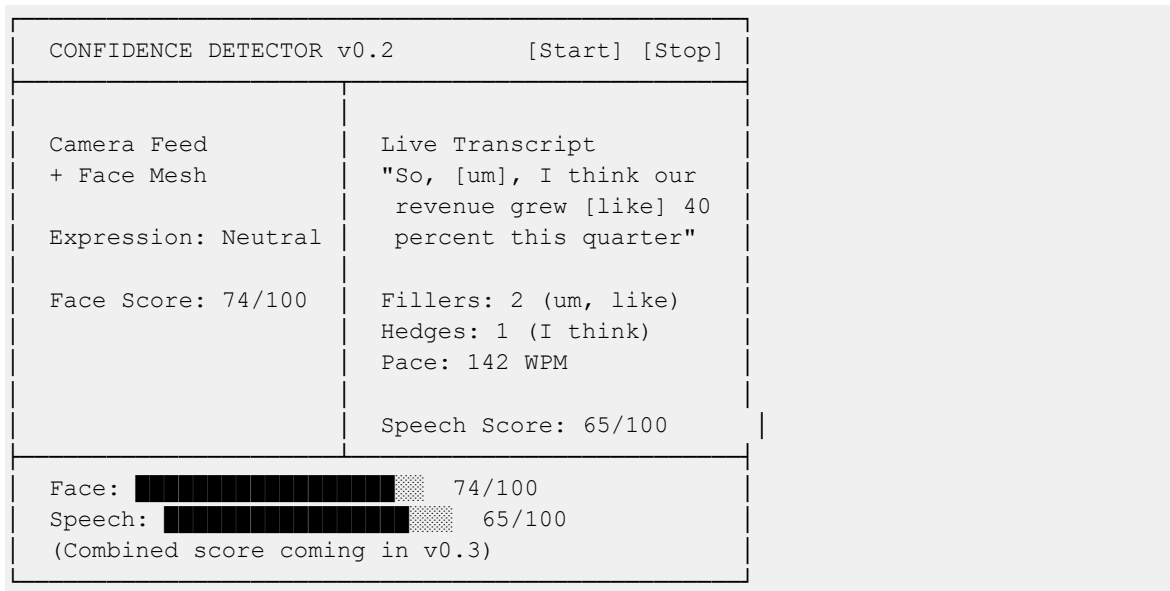
- Initialize Web Speech API (`webkitSpeechRecognition`)
- Handle continuous recognition (auto-restart on silence)
- Capture interim results (words appearing as you speak)
- Capture final results (complete sentences)
- Store full transcript with timestamps

Step 3: Add NLP Analysis Engine (4-8 hours)

Wire up Phase 4 skills:

- Tokenize each completed sentence
- Run filler word detection → count and highlight
- Run hedging phrase detection → count
- Calculate speaking pace (WPM) using word count + time
- Run repetition detection
- Calculate speech text score (0-100)

Step 4: Build the Two-Panel UI (4-8 hours)



Key UI features:

- Left panel: camera feed with mesh overlay + face score
- Right panel: scrolling transcript with colored filler highlights
- Bottom: score bars for each engine
- Filler words highlighted in orange/red in the transcript
- Real-time WPM counter

Step 5: Performance Check (2-4 hours)

Verify both engines run smoothly together:

- Face mesh still hits 15+ FPS with STT running
- Transcript updates do not cause face mesh to stutter
- No memory leaks during 10-minute sessions
- Profile with Chrome DevTools, fix bottlenecks

Success Criteria

Metric	Target
Face mesh + expression	Still works from v0.1
Transcript appears live	Words show as you speak
Filler words highlighted	Orange/red color on fillers
Face score displayed	0-100, updates in real time

Speech score displayed	0-100, updates per sentence
Both engines run simultaneously	No freezing, no lag
WPM counter	Shows current speaking pace
10-minute session	No crashes, no memory leaks

Limitations (What v0.2 Cannot Do)

Cannot do: Combined confidence score, voice pitch analysis, coaching alerts, session reports, AI feedback, persistent cross-session tracking

The two scores are shown SEPARATELY — no combined score yet. That is intentional. Verify each engine works before combining.

After You Build This

You now have a two-engine system that reads faces AND listens to speech. The transcript with highlighted fillers is already a useful tool on its own.

You have also proven that two engines can run simultaneously in the browser. This gives you confidence (pun intended) that adding engines 3 and 4 in v0.3 is feasible.

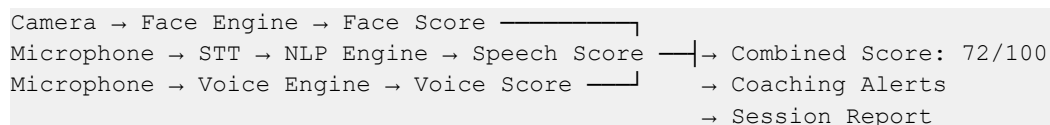
Move on to v0.3 to add voice analysis and combine everything.

Version 0.3 — Full Detection

BUILD THIS AFTER: Phase 5 (System Integration, Sprints 1-6)

What This Version Does

All engines running. One combined score. Real-time dashboard with coaching alerts. Session report.



This is the v1 demo. This is the version you show to corporate training managers and say "This is what the Presentation Confidence Detector does."

Why This Is the Critical Version

v0.1: "Look, it tracks my face" → Tech demo
v0.2: "Look, it listens too" → Two tech demos
v0.3: "Look, it coaches me in real time → PRODUCT and tells me my confidence score"

v0.3 is where the project goes from "cool experiment" to "useful product." The combined score, the coaching alerts, and the session report are what make this a tool people would actually use.

Tech Stack

Component	Technology	Purpose
Camera input	getUserMedia API	Access webcam
Face detection	MediaPipe Face Mesh	Face mesh + expressions
Microphone	getUserMedia (audio)	Access microphone
Speech-to-Text	Web Speech API	Live transcript (Chrome/Edge MVP)
NLP Analysis	Custom JS (Phase 4)	Text-based confidence signals
Voice Analysis	Web Audio API	Pitch, volume, tremor
Scoring Engine	Custom JS (Phase 5)	Weighted score combination
Coaching System	Custom JS (Phase 5)	Rule-based alerts + cooldown
Session Recorder	Custom JS (Phase 5)	Time-series data storage
Report Generator	React components	Post-session summary + charts
Charts	Chart.js or Recharts	Score timeline, breakdowns
UI Framework	React + Vite	Full dashboard

What Phases Must Be Complete

Phase	Required?	Why
Phase 1: Hardware Access	YES	Camera + microphone access
Phase 2: Speech & Audio	YES	STT + Web Audio API for voice analysis
Phase 3: Face & Expression	YES	MediaPipe Face Mesh + expression scoring
Phase 4: NLP Text Analysis	YES	All text analysis functions
Phase 5: System Integration	YES (Sprints 1-6)	Multi-engine orchestration, scoring, dashboard, alerts, recording, reports

What You Will Learn Building This

14. **Four-engine orchestration** — Running face, STT, NLP, voice simultaneously
15. **Scoring algorithm** — Weighted combination of 3 scored outputs into one number
16. **Real-time dashboard** — Animated gauge, live charts, waveform display
17. **Coaching system** — Rule-based alerts with cooldown logic
18. **Session management** — Start, record, stop, generate report lifecycle
19. **Report generation** — Aggregating time-series data into a readable summary
20. **Performance under load** — Keeping 15+ FPS with all engines running
21. **Error recovery** — What happens when one engine fails mid-session?

Step-by-Step Build Guide

Step 1: Add Voice Analysis Engine (2-3 days)

Build the voice engine using Web Audio API:

- Create AudioContext and AnalyserNode
- Extract pitch (fundamental frequency) using autocorrelation
- Extract volume (RMS of audio buffer)
- Detect vocal tremor (pitch variance over short window)
- Calculate voice score (0-100): steady pitch + good volume + low tremor = high score

Step 2: Build the Scoring Engine (1-2 days)

Implement the combined scoring algorithm:

- Accept face_score, speech_score, voice_score as inputs
- Start with weights such as 40% face, 35% speech, 25% voice
- Handle missing scores (if one engine is off, redistribute weights)
- Apply exponential moving average for smooth score transitions
- Output a single 0-100 combined confidence score

Step 3: Build the Dashboard UI (3-5 days)

Create the full dashboard layout:

- Confidence gauge (large, animated, center of attention)
- Camera feed with face mesh (top left)
- Live transcript with highlighted fillers (right panel)
- Sub-score bars (face, speech, voice)
- WPM counter
- Audio waveform display
- Score timeline chart (line graph updating in real time)
- Session timer
- Start / Stop / Reset buttons

Step 4: Build the Coaching Alert System (1-2 days)

Implement rule-based coaching:

- Define alert rules (pace too fast, low eye contact, too many fillers, etc.)
- Implement cooldown timers (do not spam the same alert)
- Priority system (show the most important alert)
- Alert UI component (slides in, fades out after 5 seconds)
- Maximum 2 active alerts at a time

Step 5: Build Session Recording (1 day)

Implement data recording:

- Sample all scores every 1 second
- Store in an array: `{timestamp, combined, face, speech, voice, pace, fillerCount}`
- Store all coaching alerts that fired (with timestamps)
- Store full transcript
- Calculate session duration

Step 6: Build Report Generation (2-3 days)

Create the post-session report:

- Overall score (average of combined score over session)
- Score timeline chart (static, showing the full session)
- Breakdown by engine (face vs. speech vs. voice)
- Key issues list (sorted by impact on score)
- Strengths list
- Top recommendation
- Filler word summary (total count, most frequent)
- Pace analysis (average WPM, variance, spikes)

Step 7: Integration Testing (2-3 days)

Test the complete system:

- Run a 10-minute practice session — does everything work?
- Test with different lighting conditions (face engine)
- Test with background noise (STT + voice engine)
- Test in Chrome, test in Edge
- Profile performance — is it above 15 FPS?
- Test error scenarios: unplug camera mid-session, mute mic
- Fix bugs, tune scoring weights, adjust alert thresholds

Success Criteria

Metric	Target
All processing engines run smoothly without freezing	Face, STT, NLP, Voice all active without
Combined confidence score	0-100, updates smoothly
Coaching alerts appear	When thresholds are crossed
Coaching alerts have cooldown	No spam (30-60 sec cooldown)
Session recording works	Data captured every 1 second
Session report generates	After pressing Stop
Report includes timeline chart	Yes
Report includes breakdown	Face / Speech / Voice scores
10-minute session	No crashes, > 15 FPS
Dashboard looks professional	Clean layout, readable fonts

Limitations (What v0.3 Cannot Do)

Cannot do: AI-powered feedback (Claude API), session history, progress tracking across sessions, persistent baseline calibration, PDF export, multi-person

These are v1.0 features.

After You Build This

This is your demo product. v0.3 is the version you show to people and say "I built a browser-first presentation confidence detector prototype with no custom backend for the core detection loops."

It has a real-time confidence score, coaching alerts, and a session report. It is genuinely useful for anyone who wants to practice presenting.

Take a moment to appreciate this: you built a multi-engine AI system with browser-based real-time detection and reporting. That is impressive.

Then move on to v1.0 for the polish that makes it feel like a real product.

Version 1.0 — Production Polish

BUILD THIS AFTER: Phase 5 complete + Phase 4 stretch goals

What This Version Does

Everything from v0.3, PLUS:

- Claude API integration (optional toggle, via backend proxy) for AI-powered feedback
- Session history with progress charts (track improvement over time)
- Persistent baseline calibration (the system learns YOUR normal across sessions)
- PDF export of session reports
- Polished UI suitable for a product demo or portfolio piece

```
v0.3 = "It works"
v1.0 = "It works AND it looks like a real product"
```

Why v1.0 Matters

v0.3 proves the technology. v1.0 proves you can build a PRODUCT. The difference:

v0.3	v1.0
Works but looks rough	Polished, professional UI
No memory between sessions	Session history + progress
Same thresholds for everyone	Calibrated to each user
Rule-based feedback only	AI-powered deep feedback
Cannot share results	PDF export

v1.0 is what goes in your portfolio. It is what you demo at interviews. It is the version that makes someone say "Wait, YOU built this?"

Tech Stack

Component	Technology	Purpose
-----------	------------	---------

Everything from v0.3	(same)	Core system
AI Feedback	Claude API via backend	Deep analysis + advice
Session Storage	localStorage / IndexedDB	Save session history
Progress Charts	Chart.js or Recharts	Improvement over time
Baseline System	Custom JS	Per-user calibration
PDF Export	html2pdf.js or jsPDF	Download reports
UI Polish	Tailwind CSS	Professional styling

What Phases Must Be Complete

Phase	Required?	Why
Phase 1-5	ALL	Everything from v0.3
Phase 4 stretch	Recommended	Advanced NLP features for richer analysis
Phase 5 Sprint 7+	YES	AI API integration, session history, calibration

What You Will Learn Building This

22. **API integration** — Calling Claude through your backend route and keeping provider secrets server-side
23. **Prompt engineering** — Writing prompts that generate useful, specific presentation feedback
24. **Client-side storage** — IndexedDB or localStorage for persisting session data
25. **Progress visualization** — Charts showing improvement across multiple sessions
26. **Calibration logic** — Recording a baseline and scoring relative to the user's normal
27. **PDF generation** — Converting HTML/canvas content to downloadable PDF
28. **UI/UX polish** — Transitions, loading states, error handling, responsive design
29. **Product thinking** — What makes a tool feel "finished" vs "a prototype"

Step-by-Step Build Guide

Step 1: Session History (2-3 days)

Save and recall past sessions:

- After each session, save the report data to IndexedDB
- Build a "Session History" page listing past sessions
- Each entry shows: date, duration, overall score, one-line summary
- Click a session to view the full report
- Build a "Progress" chart: overall score over time (line chart, one point per session)
- Show trend: "Your average score improved from 58 to 72 over 8 sessions"

Step 2: Baseline Calibration (2-3 days)

Personalize the scoring:

- On first use, prompt: "Let us calibrate. Speak naturally for 30 seconds."

- During calibration, record: normal pace (WPM), normal pitch range, normal filler rate, resting expression
- Store the baseline in localStorage or IndexedDB so it persists across sessions
- Adjust scoring relative to baseline:
- If the user's normal pace is 160 WPM, then 160 is not "too fast" for THEM
- If the user naturally says "like" a lot, weight fillers slightly lower
- Show "Compared to your baseline: +12 points" on reports

Step 3: Claude API Integration (3-5 days)

Add AI-powered deep feedback safely:

- Add an optional "AI Feedback" toggle in settings
- Add a small backend/proxy route that calls Claude API and keeps the provider key out of the browser
- After a session ends (or on demand during a session), send data from the browser to your backend route:
- Transcript excerpt (last 2 minutes or full session summary)
- Score breakdown (face, speech, voice)
- Key metrics (filler count, pace, eye contact ratio)
- Prompt Claude for:
- 3 specific things to improve (with examples from the transcript)
- 2 strengths observed
- One concrete exercise to practice
- Display AI feedback in a dedicated section of the report
- Handle API errors gracefully (if API is down, show rule-based feedback only)
- Keep API keys server-side. If you support user-provided keys for advanced local use, send them to the backend only and never embed them in frontend code.

Step 4: PDF Export (1-2 days)

Let users download their reports:

- Add "Export PDF" button on the session report page
- Use html2pdf.js to convert the report div to PDF
- Include: scores, timeline chart, breakdown, key issues, AI feedback (if enabled)
- Format for readability: headers, spacing, charts render correctly
- File name: `confidence-report-2026-03-15.pdf`

Step 5: UI Polish (3-5 days)

Make it look professional:

- Consistent color scheme (pick a palette and stick to it)
- Smooth transitions on score changes (CSS transitions)
- Loading states for every async operation
- Error states with helpful messages (not just "Error")
- Responsive layout (works on different screen sizes)
- Settings page (toggle AI feedback, adjust alert preferences, manage API key)

- Onboarding flow for first-time users (calibration prompt, feature tour)
- About/Help page explaining how the scoring works
- Favicon + page title + meta tags

Step 6: Final Testing (2-3 days)

Full end-to-end testing:

- Complete a 10-minute session with all features enabled
- Review the session report — does it make sense?
- Check AI feedback — is it specific and useful?
- Export PDF — does it look good?
- Check session history — are past sessions saved correctly?
- Check progress chart — does it show improvement?
- Test baseline calibration — does personalized scoring feel more accurate?
- Test in Chrome and Edge first
- Test Firefox only if you switch STT away from Web Speech API to an option with Firefox support
- Test with slow internet (API calls)
- Test with no internet (graceful degradation)
- Performance check: still 15+ FPS with everything running

Success Criteria

Metric	Target
Everything from v0.3	Still works
Session history persists	Survives browser refresh
Progress chart shows trend	Visible improvement over sessions
Baseline calibration works	Personalized scoring feels accurate
Claude API feedback	Specific, actionable, useful
PDF export	Clean, readable, includes charts
UI looks professional	Someone would believe this is a product
First-time onboarding	New user can start in < 2 minutes
Graceful degradation	Works without API, works without mic

After You Build This

You have a portfolio-quality product.

What you built:

- A multi-engine AI system running primarily in the browser
- Real-time computer vision (face mesh + expression detection)
- Real-time speech-to-text with NLP analysis
- Real-time voice analysis (pitch, volume, tremor)
- A scoring algorithm that combines 3+ signals into one number
- A coaching alert system with cooldown logic
- Session recording with timeline visualization
- AI-powered feedback via Claude API through a backend proxy

- Session history with progress tracking
- Baseline calibration for personalized scoring
- PDF export
- A polished, professional UI

This is not a tutorial project. This is a real product that solves a real problem. It belongs in your portfolio, on your resume, and in your interview stories.

Risk Assessment Per Version

Every version has one thing most likely to go wrong. Know it before you start building.

Version	Biggest Risk	Mitigation
v0.1 Face Only	MediaPipe JS may be slow in browser on old devices	Test on 3 different devices. If <10 FPS, use lower resolution (640x480 instead of 1280x720)
v0.2 Face+Speech	Web Speech API is Chrome-only and has a 60-second timeout	Auto-restart pattern in <code>onend</code> handler. Document browser requirement clearly. Accept Chrome/Edge only for MVP
v0.3 Full Demo	All 4 engines running simultaneously may freeze the UI	Use Web Workers for heavy processing. Process face every 3rd frame if needed. Profile with DevTools before adding features
v1.0 Production	Claude API costs, session storage limits, PDF generation complexity	API key is optional (rule-based feedback works without it). LocalStorage for sessions (IndexedDB if data grows). html2pdf.js for export (proven library, minimal setup)

The pattern: Each version's risk comes from adding the new thing, not from the things that already work. That is why we build in layers. v0.1 proves face works before v0.2 adds speech. v0.2 proves two engines work before v0.3 adds the rest.

Testing Strategy Per Version

Do not wait until v1.0 to test. Each version has specific things to verify.

Version	What to Test	How
v0.1	Does face mesh load? 15+ FPS? Blink detection works?	Record yourself for 60 seconds. Verify FPS counter stays above 15. Blink rapidly — blink count should increase. Smile — expression label should change to "Happy"

v0.2	Does STT transcribe correctly? Fillers detected?	Say "um" 5 times on purpose — filler count should be 5. Say "I think maybe" — hedge count should be 2. Check WPM counter against a known-pace reading
v0.3	Combined score feels right? Dashboard responsive?	Present confidently (strong voice, eye contact, no fillers) — score should be 70+. Present nervously (look away, say "um", speak quietly) — score should be <40. Switch between confident and nervous — score should change within 5 seconds
v1.0	Session save/load? PDF exports? Claude API response?	Complete 3 sessions. Close browser. Reopen — sessions should still be there. Export PDF — verify content matches what you see on screen. Toggle AI feedback on — verify Claude response appears in report

Why these specific tests matter: They catch the most common failures. FPS drops mean MediaPipe is too heavy. Filler count mismatches mean the NLP tokenizer has a bug. Score not changing means the smoothing factor is too aggressive. These are the bugs you WILL hit.

What If Something Takes Longer?

Not everything will work on the first try. Here is the branching plan for the most likely blockers.

If face mesh is too slow:

Switch from MediaPipe Face Mesh (468 landmarks) to face-api.js. Fewer features (no blendshapes), but lighter weight. You lose detailed expression mapping but keep basic emotion detection. Acceptable for MVP.

If Vosk accuracy is bad:

Switch to Web Speech API for the MVP. It is Chrome-only, which is acceptable for a demo. Vosk can be added back later when you have time to tune the model. The NLP analysis layer does not care where the transcript came from.

If all 4 engines cannot run together:

Drop the voice engine first. It has the lowest weight (0.25), and pitch/volume analysis is the hardest to get right. Face (0.40) + Speech (0.35) already give you 75% of the score. Ship the 2-engine version, add voice back when performance allows.

If scoring feels wrong:

Stop adding features. Spend 2 days recording yourself presenting in different styles (confident, nervous, monotone, energetic). Compare the scores to your gut feeling. Tune the weights and thresholds until the score matches your intuition. A wrong score undermines the entire product.

The rule: When blocked, cut scope, do not cut quality. A 2-engine system that scores accurately is better than a 4-engine system that gives wrong numbers.

Version Summary

Version	What It Proves	Engines	Time to Build	Demo Value
v0.1	MediaPipe works in browser	Face only	3-5 days	"Look, it tracks my face"
v0.2	Two engines can run together	Face + STT + NLP	1 week	"It reads my face AND speech"
v0.3	Full detection system works	Face + STT + NLP + Voice	2-3 weeks	"Real-time confidence coaching"
v1.0	Production-quality product	All + AI API + History	2 weeks	"I built an AI speech coach"